Aman_Views

Cloud Security Notes

1. Cloud Security Fundamentals

Cloud Security refers to the policies, technologies, controls and best practices designed to protect data, applications and infrastructure in the cloud.

Shared Responsibility Model

Cloud Provider (Security of the Cloud)	Customer (Security in the Cloud)
• Physical datacenters	• Data classifications & protection
• Hardware, servers, storage	• Identity & access management
• Networking infrastructure	• OS, applications & patching
• Virtualization layer	• Network configuration
• Managed services	• Security groups & firewalls
• Availability of cloud services	• Monitoring, logging & compliance

IaaS vs PaaS vs SaaS Security

Model	Customer Manages	Provider Manages
IaaS	OS, Applications, Data, Middleware, Network config, IAM	Physical Infra, Hardware, Virtualization, Storage, Networking
PaaS	Applications, Data, IAM, Configurations	Runtime, Middleware, OS, Servers, Storage, Networking
SaaS	Data, Users, Access Policies	Applications, Runtime, OS, Servers, Storage, Networking

CIA Triad



Common Cloud Security Risks

- Data Breaches
- Misconfigurations
- Account Hijacking
- Insecure APIs
- Insufficient IAM
- Data Loss
- Denial of Service (DoS)
- Shared Technology Vulnerabilities
- Insider Threats
- Malware & Ransomware
- Weak Encryption / Key Management

Cloud Security vs Traditional Security

Cloud Security	Traditional Security
• Shared responsibility	• Full responsibility by organization
• Elastic & dynamic environments	• Static environments
• Internet exposed by default	• Perimeter-based security
• API-driven management	• Manual management
• Multi-tenant architecture	• Single-tenant architecture
• Continuous monitoring required	• Periodic monitoring

Security Controls in Cloud

- Preventive Controls → Stop threats from occurring.
- Detective Controls → Detect and alert on threats.
- Corrective Controls → Respond and fix issues.
- Administrative Controls → Policies, training, awareness.
- Technical Controls → Firewalls, encryption, IAM, etc.
- Physical Controls → Physical security of datacenters.

Key Takeaways

- ☑ Cloud security is a shared responsibility.
- ☑ Customer is responsible for security "in" the cloud.
- ☑ Understand IaaS, PaaS and SaaS to know your responsibility.
- ☑ Protect Confidentiality, Integrity and Availability of data.
- ☑ Identify risks and implement strong security controls.

Cloud Security Principles

- Least Privilege
- Defense in Depth
- Zero Trust
- Encryption Everywhere
- Continuous Monitoring
- Automation & Compliance

Remember

The cloud is powerful, but security is your responsibility.
Secure the things you build, deploy and manage.

@aman_views

Cloud Security Notes

2. Identity & Access Management (IAM)

IAM ensures the right people (or services) have the right access to the right resources for the right reasons.

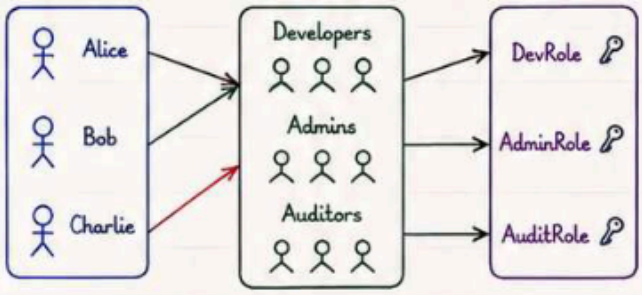
Authentication vs Authorization

Authentication	Authorization
Verifies who you are. (Identity Verification)	Determines what you are allowed to do.
Ex: Password, MFA, Biometric, OTP	Ex: Permissions, Policies, Roles

Key IAM Components

- **Users** – Individual human users who access resources.
- **Groups** – Collection of users for easy management.
- **Roles** – Set of permissions that can be assigned to users or services.
- **Policies** – Rules that define what actions are allowed or denied.
- **Permissions** – Specific actions allowed on a resource.
- **Identities** – Human users or machine/service identities.

Users, Groups & Roles



Policies & Permissions

Example Policy (JSON)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject"],
      "Resource": ["arn:aws:s3:::my-bucket/*"]
    }
  ]
}
```

Principle of Least Privilege

Grant only the minimum level of access required to perform a task. No more, no less.

RBAC (Role-Based Access Control)

Component	Description	Benefit
Role	A role is a collection of permissions.	Simplifies access management.
User	Users are assigned one or more roles.	Easy to manage at scale.
Permission	Specific actions allowed on resources.	Centralized control.
Access	Users get access based on assigned roles.	Reduces risk of over-permission.

Service Accounts

- Used by applications, services, and workloads to access cloud resources.
- Have unique credentials.
- Should have scoped-down permissions.
- Rotate keys regularly.

Human vs Machine Identities

Human Identities	Machine Identities
• Managed by humans. • Login via password / MFA. • Example: Developers, Admins. • More prone to phishing.	• Used by applications & services. • Managed via service accounts / roles. • No human login. • Example: EC2 instance, Lambda.

Common IAM Mistakes

- ✗ Using root account for daily tasks.
- ✗ Over-permissioned users and roles.
- ✗ Hard-coded credentials in applications.
- ✗ Not using MFA.
- ✗ Not rotating access keys.
- ✗ Sharing accounts instead of using roles.
- ✗ Ignoring unused users and permissions.

Remember

Strong IAM is the first line of defense in the cloud. Secure identities, secure everything!

@aman_views

Cloud Security Notes

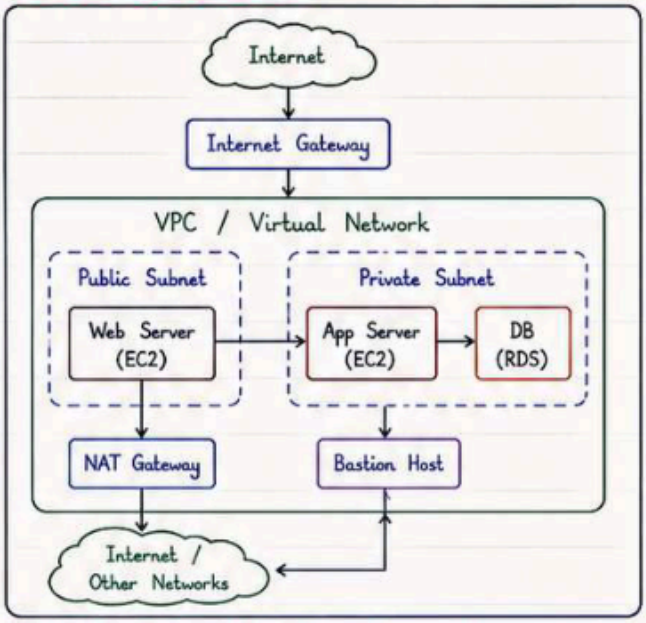
3. Cloud Network Security

Cloud network security focuses on protecting data in transit, controlling traffic and isolating resources in the cloud.

Core Network Components

① VPC / Virtual Network — Isolated virtual network in the cloud.
② Subnets — Divide the network into smaller segments.
③ Internet Gateway — Enables communication between VPC and internet.
④ NAT Gateway — Allows private instances to access internet securely.
⑤ Route Table — Controls where network traffic is directed.
⑥ Security Groups — Stateful firewall for instances.
⑦ Network ACLs — Stateless firewall for subnets.
⑧ Bastion Host — Secure access point to private resources.

Network Traffic Flow



Security Groups (Stateful Firewall)

Type	Direction	Description
Inbound Rules	Traffic coming into the instance.	Allow only required ports (e.g., 80, 443, 22).
Outbound Rules	Traffic going out from the instance.	Allow required traffic to external services.

Network ACLs (Stateless Firewall)

Type	Direction	Description
Inbound Rules	Applied at subnet level – Inbound.	Allow or deny traffic based on rules.
Outbound Rules	Applied at subnet level – Outbound.	Evaluated in order (numbered rules).

Inbound vs Outbound Traffic

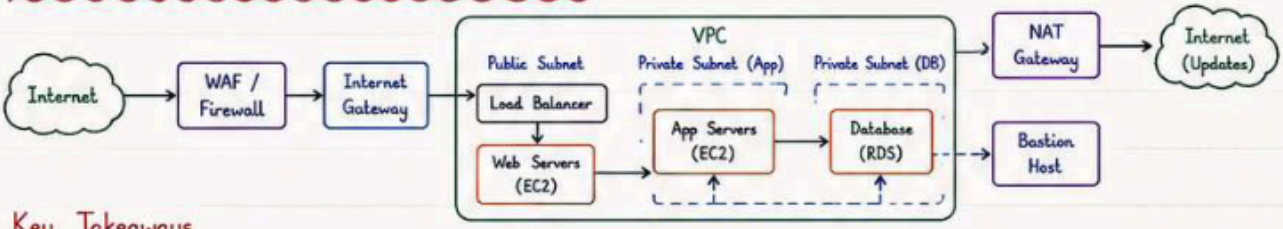
Inbound Traffic
- Entering the resource from external sources.
- Controlled using Security Groups & NACLs.
- Example: User accessing web server on port 443.

Outbound Traffic
- Leaving the resource to external destinations.
- Controlled using Security Groups & NACLs.
- Example: Instance accessing internet for updates.

Network Segmentation

- Divide network into multiple subnets.
- Isolate public-facing resources from private resources.
- Use separate subnets for Web, App and Database tiers.
- Reduce blast radius in case of security breach.
- Apply least privilege between segments.

Secure Cloud Network Architecture (Example)



Key Takeaways

- ☑ Use VPC to isolate your cloud resources.
- ☑ Keep private resources in private subnets.
- ☑ Allow only necessary traffic using Security Groups & NACLs.
- ☑ Use NAT Gateway for secure outbound internet access.
- ☑ Access private servers only via Bastion Host.
- ☑ Implement network segmentation for better security.

Remember

💡 Good network design is the foundation of cloud security. Isolate, control and monitor every connection!

@aman_views

Cloud Security Notes

4. Data Security & Encryption

Protecting data is a critical part of cloud security. Data can be at rest, in transit or in use. Encryption ensures confidentiality and integrity.

Data at Rest vs Data in Transit

Data at Rest	Data in Transit
• Data stored in databases, storage, backups, files. • Example: S3 bucket object, EBS volume. • Protect using encryption.	• Data moving between systems over a network. • Example: API requests, file uploads. • Protect using TLS/HTTPS.

Symmetric vs Asymmetric Encryption

Symmetric Encryption	Asymmetric Encryption
• Same key for encryption and decryption. • Faster, good for large amounts of data. • Example: AES, DES	• Different keys: Public key to encrypt, Private key to decrypt. • Slower, used for secure key exchange. • Example: RSA, ECC

Encryption Keys

- Keys are used to encrypt and decrypt data.
- Strong keys = Strong security.
- Keys must be stored securely.
- Never hardcode keys in applications.
- Limit key access to only authorized identities.

Key Management Service (KMS)

- Managed service to create, store and manage encryption keys.
- Provides hardware security module (HSM) for key protection.
- Controls key usage through IAM policies.
- Example: AWS KMS, Azure Key Vault, GCP Cloud KMS.

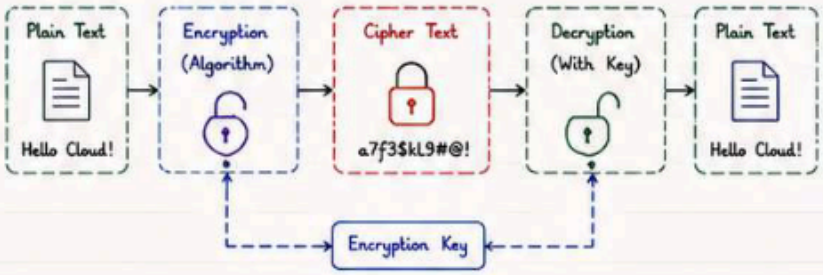
Key Rotation

- Regularly changing encryption keys.
- Reduces risk if a key is compromised.
- Automate rotation where possible.
- Old keys should be securely archived or destroyed.

Secrets vs Encryption Keys

Secrets	Encryption Keys
• Passwords, API keys, tokens, certificates. • Accessed by applications directly. • Managed using Secrets Manager / Vault.	• Used to encrypt / decrypt data. • Managed using KMS / Key Vault. • Require strict access controls.

Data Encryption Overview



Where Encryption is Used

Databases | Object Storage | Block Storage | Backups | Emails | APIs | Mobile Apps

Database Encryption

- Encrypt database storage.
- Use Transparent Data Encryption (TDE).
- Encrypt sensitive columns at application layer.
- Restrict database access using security groups.

Storage Encryption

- Enable default encryption (SSE).
- Use customer managed keys (CMK) for more control.
- Enable bucket / disk level encryption.
- Enforce encryption using policies.

TLS / HTTPS (Data in Transit)

- Use TLS to encrypt data in transit.
- Enforce HTTPS for all web traffic.
- Use strong TLS versions (TLS 1.2 / 1.3).
- Disable weak ciphers and protocols.

Common Data Security Mistakes

✗ Storing data in plain text.	✗ Hardcoding keys in application.	✗ Using weak encryption.	✗ Not rotating encryption keys.	✗ Publicly exposing sensitive data.

Remember

Encrypt your data. Manage your keys. Protect your secrets.
Strong data security builds customer trust and protects your cloud environment.

@aman_views

Cloud Security Notes

5. Cloud Storage Security

Cloud storage services are highly scalable and durable, but misconfigurations can lead to major data exposure.

Object Storage Security

- Stores data as objects in buckets.
- Highly durable and available.
- Accessed over the internet via APIs.
- Examples: AWS S3, Azure Blob, GCP Cloud Storage.

Bucket Permissions

Permission	Description
List Bucket	Allows listing of objects in the bucket.
Read Object	Allows downloading object data.
Write Object	Allows uploading or modifying objects.
Delete Object	Allows deleting objects.

Public vs Private Storage

Public Storage	Private Storage
• Accessible by anyone on the internet. • Easy to share. • Risk of data leakage if misconfigured.	• Accessible only with explicit permissions. • More secure by default. • Recommended for sensitive data.

Access Policies

Policy Type	Description
Bucket Policy	Attached to bucket. Controls access to all objects in bucket.
Object ACL	Access control at individual object level.
IAM Policy	Controls access based on users/roles.

Pre-signed URLs

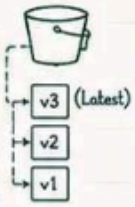
- Generate a temporary URL to access an object.
- Has expiry time and limited permissions.
- Useful for secure sharing without making object public.
- Revoke or expire URLs to stop access.

Encryption

Type	Description
Server-Side Encryption (SSE)	Data encrypted at rest by cloud provider.
Client-Side Encryption	Data encrypted before uploading by client.
SSE-KMS / CMK	Use customer managed keys for encryption.

Versioning

- Keeps multiple versions of an object.
- Protects against accidental deletion or overwrite.
- Enables easy recovery.



Access Logging

- Enable access logs to track requests to bucket.
- Helps in monitoring and troubleshooting.
- Store logs in separate bucket.



Data Leakage

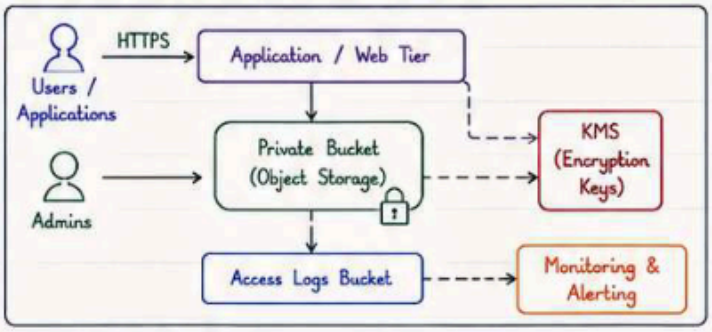
- Misconfigured permissions.
- Public buckets.
- Exposed credentials.
- Overly permissive policies.



Preventing Accidental Public Exposure

- ☑ Block public access at account level.
- ☑ Use private buckets by default.
- ☑ Regularly review bucket policies and permissions.
- ☑ Enable versioning and MFA delete.
- ☑ Use least privilege IAM roles.
- ☑ Enable access logging and monitoring.
- ☑ Scan for public buckets using cloud security tools.

Secure Storage Architecture (Example)



Key Takeaways

- ☑ Cloud storage is secure when configured properly.
- ☑ Least privilege access and encryption are essential.
- ☑ Monitoring and logging help in early detection.
- ☑ Always review and test your storage permissions.

Common Mistakes

- ✗ Making buckets public without need.
- ✗ Using "*" in bucket policies.
- ✗ Not enabling encryption.
- ✗ Ignoring access logs and monitoring.

Remember

A single misconfigured bucket can expose terabytes of sensitive data. Secure it, monitor it, protect it!

Cloud Security Notes

6. Cloud Application & API Security

Cloud applications and APIs are major attack targets. Secure design, proper authentication and controls help protect data and services.

API Authentication

- Verifies who is calling the API.
- Methods: API Key, OAuth 2.0, JWT, Mutual TLS.
- Always authenticate before processing any request.

API Authorization

- Verifies what the caller is allowed to do.
- Use roles, permissions and scopes.
- Enforce least privilege access.

OAuth 2.0 (Authorization Framework)

- Industry standard for delegated access.
- Key flows: Authorization Code, Client Credentials, Refresh Token.
- Used widely for user and service access.

JSON Web Token (JWT)

- Compact, URL-safe token format.
- Contains: Header, Payload, Signature.
- Used for stateless authentication.
- Always verify signature and expiry.

API Gateway

- Single entry point for APIs.
- Handles routing, auth, throttling, logging, transformations.
- Enforces security policies and reduces backend exposure.

Rate Limiting

- Limit number of requests per user/IP.
- Prevents abuse and brute force.
- Types: Per second, Per minute, Per day limits.

Input Validation

- Validate all inputs (type, length, format, range).
- Prevent injection, XSS, command injection.
- Never trust user input.

CORS (Cross-Origin Resource Sharing)

- Controls which domains can access your API.
- Configure allowed origins, methods and headers.
- Avoid using wildcard (*) in production.

Web Application Firewall (WAF)

- Filters and monitors HTTP/HTTPS traffic.
- Protects from common attacks: SQLi, XSS, RFI, LFI, Bot attacks.
- Deployed in front of web apps/APIs.

API Keys

- Simple way to identify applications or users.
- Send in headers or query params.
- Rotate keys regularly.
- Do not hardcode in client-side apps.

Service-to-Service Security

- Use mutual TLS or OAuth Client Credentials for service authentication.
- Use short-lived tokens.
- Encrypt traffic between services.
- Restrict permissions to only required resources.
- Monitor service accounts and access logs.

Common Cloud API Attacks

Attack	Description	Impact	Prevention
Broken Authentication	Weak or missing authentication.	Unauthorized access.	Strong auth, MFA, token validation.
Broken Authorization	Improper access control.	Access to sensitive data/functions.	Use RBAC, scopes, least privilege.
Injection Attacks	SQLi, NoSQLi, Command injection.	Data breach, system compromise.	Input validation, parameterized queries.
Excessive Data Exposure	APIs return more data than needed.	Data leakage.	Return only required data.
Broken Object Level Auth	No check on object ownership.	Access others' data.	Verify ownership and permissions.
Rate Limit Abuse	APIs abused by high requests.	DoS, service disruption.	Apply rate limiting and throttling.
Security Misconfiguration	Weak CORS, open endpoints.	Unauthorized access.	Secure configurations & reviews.
Sensitive Data in Logs	Logging tokens, PII, passwords.	Data exposure.	Mask sensitive data in logs.

Key Takeaways

- ☑ Secure APIs from design to deployment.
- ☑ Authenticate, authorize and validate always.
- ☑ Use gateways, WAF and rate limiting.
- ☑ Encrypt data and protect tokens.
- ☑ Monitor and log API activities.

Remember

→ APIs are the backbone of cloud applications. Secure APIs = Secure Cloud.

Cloud Security Notes

7. Containers, Kubernetes & Serverless Security

Modern cloud applications use containers, Kubernetes and serverless functions. Securing them requires special attention.

Container Security

- Containers share the host OS kernel.
- If one container is compromised, others may be at risk.
- Follow least privilege and isolation.
- Keep base images minimal.
- Scan images for vulnerabilities.

Image Vulnerabilities

- Vulnerabilities in base images can expose the container.
- Regularly scan images using tools (Trivy, Clair, Anchore).
- Use official and trusted images.
- Keep images updated.

Container Registries

- Store container images securely in private registries.
- Enable access controls and MFA.
- Scan images before pushing.
- Sign images to ensure integrity.

Secrets in Containers

- Never store secrets in images.
- Use environment variables or secrets management tools.
- Use Kubernetes Secrets or cloud secret managers.
- Rotate secrets regularly.

Kubernetes RBAC

- Use Role-Based Access Control.
- Create least privilege roles.
- Do not use cluster-admin for regular users.
- Regularly review and audit roles and bindings.

Kubernetes Secrets

- Store sensitive data in Kubernetes Secrets (base64 encoded).
- Enable encryption at rest.
- Restrict access using RBAC.
- Consider external secret stores (AWS Secrets Manager, Vault).

Network Policies

- Control traffic between pods.
- Deny all by default and allow only necessary communication.
- Prevent lateral movement.
- Use namespace isolation.

Pod Security

- Run containers as non-root users.
- Drop unnecessary capabilities.
- Use read-only file systems.
- Apply security contexts and Pod Security Standards.

Serverless Security

- Functions run with limited privileges.
- Use least privilege IAM roles.
- Validate inputs and sanitize data.
- Protect environment variables and secrets.

Function Permissions

- Grant only required permissions to functions.
- Avoid wildcard (*) permissions.
- Use resource-based policies.
- Regularly review permissions.

Dependency Vulnerabilities

- Application dependencies may contain vulnerabilities.
- Use dependency scanning tools.
- Keep libraries and frameworks updated.
- Remove unused dependencies.

Secure Deployment Pipeline

- Scan code, dependencies and images.
- Use automated security checks in CI/CD.
- Sign artifacts and images.
- Deploy to isolated environments first (Dev → Staging → Prod).

Container & Kubernetes Security Checklist

- ☑ Use trusted base images.
- ☑ Scan images for vulnerabilities.
- ☑ Do not run containers as root.
- ☑ Use least privilege for users and services.
- ☑ Store secrets securely (not in images).
- ☑ Apply network policies.
- ☑ Enable RBAC and limit access.
- ☑ Encrypt secrets and data.
- ☑ Monitor containers and pods.
- ☑ Keep Kubernetes and components updated.

Key Takeaways

- → Containers are isolated but not invincible.
- → Kubernetes security depends on proper configuration.
- → Follow least privilege, scan continuously and automate security in pipeline.

Remember



Security in containers, Kubernetes and serverless is all about isolation, least privilege, continuous scanning and secure automation.

Cloud Security Notes

8. Cloud Monitoring, Logging & Threat Detection

Continuous monitoring and logging help detect threats early and respond quickly.
You can't secure what you can't see.

Cloud Audit Logs

- Record API calls and management events.
- Track changes in IAM, resources and configurations.
- Example: AWS CloudTrail, Azure Activity Log.

Application Logs

- Logs from applications and services.
- Track user actions, errors and performance.
- Helps in troubleshooting and incident analysis.

Network Logs

- Capture network traffic and flow data.
- Track allowed/denied traffic.
- Example: VPC Flow Logs, NSG Flow Logs.

IAM Activity Monitoring

- Monitor user logins, role changes, permission updates.
- Detect unusual or suspicious activities.
- Helps prevent account compromise.

Security Monitoring

- Continuous monitoring of resources, workloads and configurations.
- Detect misconfigurations, weak policies and threats.
- Use dashboards and alerts.

SIEM (Security Info & Event Management)

- Collects logs from multiple sources.
- Correlates events to detect incidents.
- Example: Splunk, Microsoft Sentinel, Elastic SIEM.

IDS / IPS

- IDS: Detects suspicious activities.
- IPS: Blocks or prevents malicious traffic.
- Network based or host based.

Security Alerts

- Triggered when suspicious activity is detected.
- Send notifications via email, SMS, Slack, etc.
- Set severity levels: Low / Medium / High / Critical.

Anomaly Detection

- Identify unusual behavior using baseline and patterns.
- Detect insider threats, brute force, data exfiltration.
- Uses ML/AI in modern tools.

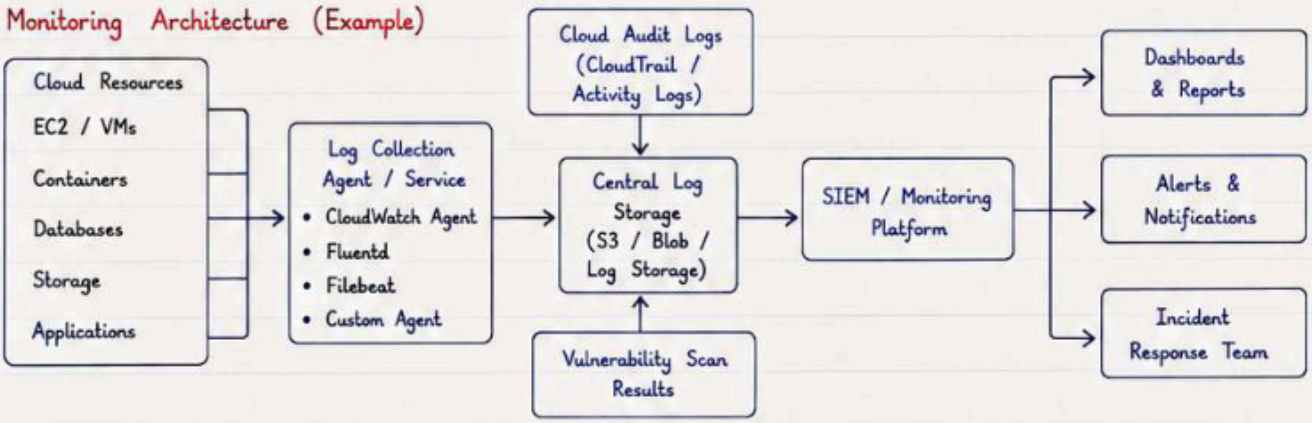
Vulnerability Scanning

- Scan cloud resources, containers, images and code.
- Find missing patches, weak configs and CVEs.
- Tools: AWS Inspector, Aqua, Trivy, Nessus.

Incident Detection Workflow

1. Collect Logs
2. Analyze Events
3. Detect Incidents
4. Alert Team
5. Respond & Mitigate
6. Review & Improve

Monitoring Architecture (Example)



Best Practices

- Enable logging for all services.
- Centralize logs in one place.
- Protect log storage (encryption, access control).
- Monitor IAM and privileged activities.
- Set meaningful alerts and thresholds.
- Regularly review logs and alerts.
- Retain logs as per compliance needs.

Common Monitoring Mistakes

- Logging is not enabled for all services.
- Alerts are not configured or ignored.
- Too many false positives.
- Logs are not reviewed regularly.
- Logs are stored without encryption.
- No retention policy for logs.

Remember

Good monitoring means early detection.

Early detection means less damage.

Monitor. Analyze. Respond. Improve.

Cloud Security Notes

9. Cloud Security Threats & Attacks

Cloud environments introduce unique threats. Understanding them is the first step towards building strong security.

1. Misconfigured Storage

- Public buckets expose sensitive data.
- Overly permissive access policies.
- No encryption or logging.
- Leads to data leakage.

2. Credential Theft

- Stealing access keys, passwords, tokens.
- Phishing attacks.
- Hardcoded credentials in code.
- Weak or reused passwords.

3. Privilege Escalation

- Gaining higher privileges than required.
- Misusing IAM roles.
- Exploiting policy weaknesses.
- Horizontal or vertical escalation.

4. Account Takeover

- Attackers take control of accounts.
- Weak MFA or no MFA.
- Stolen sessions or tokens.
- Leads to full account compromise.

5. Cloud Malware

- Malware hosted in cloud storage or instances.
- Infects systems and spreads.
- Backdoors and persistence.
- Affects workloads and data.

6. Cryptojacking

- Attackers use cloud resources to mine crypto.
- Consumes CPU/GPU heavily.
- Increases cost.
- Slows down workloads.

7. API Attacks

- Broken authentication and authorization.
- Excessive data exposure.
- Injection attacks.
- Abuse of rate limits.
- Mass assignment flaws.

8. Insider Threats

- Malicious or negligent insiders.
- Data theft or sabotage.
- Excessive access rights.
- Lack of monitoring.

9. Supply-Chain Attacks

- Compromise via third-party libraries or services.
- Vulnerable dependencies.
- Malicious updates.
- Affects entire system.

10. Data Exfiltration

- Unauthorized copying of sensitive data.
- Via APIs, storage, malware.
- Large data downloads.
- Leads to compliance issues.

11. DDoS (Denial of Service)

- Overwhelming cloud resources or APIs.
- Makes services unavailable.
- Impacts availability and reputation.
- Can be costly.

12. SSRF & Metadata Service Attacks

- SSRF targets internal services.
- Access to metadata service (e.g., 169.254.169.254).
- Steals credentials and configuration.

13. Common Cloud Security Mistakes

- Using default configurations.
- Exposing storage publicly.
- Over-permissive IAM policies.
- No encryption for data.
- Hardcoding secrets and keys.
- Lack of logging and monitoring.
- Not patching systems regularly.
- Shared responsibility confusion.

14. Prevention Strategies

- Follow least privilege principle.
- Enable MFA everywhere.
- Encrypt data at rest and in transit.
- Use IAM roles and conditions.
- Regularly review and audit access.
- Enable logging, monitoring and alerts.
- Keep systems and dependencies updated.
- Scan for vulnerabilities and misconfigurations.
- Implement network segmentation and firewalls.
- Educate teams and enforce security best practices.

Threat Impact Overview

Threat	Impact	Example	Prevention
Misconfigured Storage	Data leakage, compliance issues	Public S3 bucket	Bucket policies, encryption, logging
Credential Theft	Unauthorized access	Phishing for keys	MFA, secret rotation, least privilege
Privilege Escalation	Full system compromise	Escalate IAM role	Least privilege, access reviews
DDoS	Service disruption	Flooding APIs	Rate limiting, auto-scaling, WAF
Data Exfiltration	Loss of sensitive data	Mass data download	Monitoring, DLP, alerts

Remember

Cloud security is not a product, it's a process.
Be proactive, stay informed and keep improving.

@aman_views

Cloud Security Notes

10. Cloud Security Tools, Frameworks & Standards

Using the right tools and following industry standards helps in building and maintaining a secure cloud environment.

Popular Cloud Security Tools

- AWS: GuardDuty, Inspector, Security Hub, Macie
- Azure: Microsoft Defender for Cloud, Sentinel
- GCP: Security Command Center, Cloud Armor
- Aqua Security
- Palo Alto Prisma Cloud
- Check Point CloudGuard
- HashiCorp Vault

Security Frameworks

- NIST Cybersecurity Framework (CSF)
- CIS Controls
- ISO/IEC 27001
- SOC 2 (Service Organization Controls)
- CSA CCM (Cloud Controls Matrix)
- COBIT
- ITIL

Compliance & Standards

- GDPR – Data protection and privacy (EU)
- HIPAA – Healthcare data protection (US)
- PCI DSS – Payment card data security
- FedRAMP – For US government cloud services
- ISO 27017 – Cloud security controls

Shared Responsibility Model

Responsibility	Cloud Provider (IaaS)	Customer
Physical Data Centers	Responsible	Not Responsible
Compute / Storage / Network	Responsible	Not Responsible
Virtualization / OS	Responsible	Responsible
Applications	Responsible	Responsible
Data	Responsible	Responsible
Identities & Access Management	Responsible	Responsible
Configuration & Patch Management	Responsible	Responsible
Monitoring & Logging	Available as a service	Responsible
Backup & Recovery	Available as a service	Responsible

Zero Trust Principles

1. Never trust, always verify.
2. Least privilege access.
3. Assume breach.
4. Verify explicitly.
5. Use micro-segmentation.
6. Monitor continuously.
7. Encrypt everything.
8. Automate enforcement.

Cloud Security Checklist

- ☐ Use strong IAM policies and least privilege.
- ☐ Enable MFA for all users.
- ☐ Encrypt data in transit and at rest.
- ☐ Keep systems, applications and dependencies updated.
- ☐ Enable logging, monitoring and alerts.
- ☐ Regularly scan for vulnerabilities and misconfigurations.
- ☐ Follow secure configuration baselines (CIS).
- ☐ Implement network segmentation and firewalls.
- ☐ Backup data regularly and test recovery.
- ☐ Use secrets manager, avoid hardcoding secrets.
- ☐ Review access permissions regularly.
- ☐ Remove unused resources and accounts.
- ☐ Enable DLP and data classification.
- ☐ Conduct security audits and penetration testing.
- ☐ Document policies and ensure compliance.
- ☐ Train teams on security best practices.

Key Takeaways

- Cloud security is a shared responsibility between provider and customer.
- Strong identity, access, encryption, monitoring and configuration are the foundation.
- Understand common threats and follow frameworks & standards.
- Security is continuous – Plan, Implement, Monitor, Improve.
- Be proactive, automate security and build a security-first culture.

Security Life Cycle

Identify → Protect
Detect → Respond
Recover → Improve

Remember: Secure today, resilient tomorrow.